

Morphological Image Processing

By Wei-gang Chen

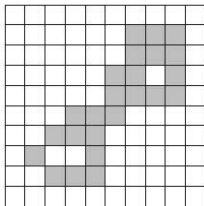
May 15, 2025

- 这一章的主要内容包括:
 - ① Fundamental definitions
 - ② Dilation and Erosion
 - ③ Opening and Closing
 - ④ 由基本运算导出的运算
 - ⑤ 灰度图像形态学滤波

- 形态学图像处理(Morphological Image Processing)
 - ① 数学形态学是一种基于集合论、拓扑学等的图像处理技术;
 - ② 主要用于分析和处理图像中的形状和结构;
 - ③ 通过**结构元素(structuring element)**与图像进行交互, 提取图像中的几何特征.

- 形态学图像处理(Morphological Image Processing)

- 在前面的内容中, 我们将图像定义为关于二维坐标的函数, 而在数学形态学中, 将图像定义为一个集合, 集合中包含连续值或离散值的坐标.
- 即该集合是某个对象的像素点的集合, 如下图中的灰色像素.



- 形态学图像处理

- ① 形态学图像处理, 其对象(如前页的灰色部分, 命名为 A)的定义:

$$O_A = \{p \mid Property(p) == \text{TRUE}\}$$

- 其中 p 是像素, 需要坐标系统标识;
- ② 相对于 A 的背景部分:

$$B_A = \{p \mid p \notin A\}$$

- ③ 通常关注的是二值图像.

- 经典形态学处理只取决于像素所在的位置, 与像素值无关.
- So, they are especially suited to the processing of binary images.

Notation From Set Theory

- 集合论的定义: 设 A 和 B 是两个集合.
 - 1 If w is an element of set A , $w \in A$.
 - 2 If w is not an element of set A , $w \notin A$.
 - 3 If set B of pixel coordinates satisfies a condition:
 $B = \{w \mid \text{conditions}\}$
 - 4 Complement of A : $A^c = \{w \mid w \notin A\}$
 - 5 Union of A and B : $A \cup B$
 - 6 Intersection of A and B : $A \cap B$

- Continue:

- ① **Difference of A and B :**

$$A - B = \{w \mid w \in A, w \notin B\} = A \cap B^c$$

- ② **集合 A 所有元素平移 $z = (z_1, z_2)$:**

$$(A)_z = \{c \mid c = a + z, \quad \text{for } a \in A\}$$

Notation From Set Theory

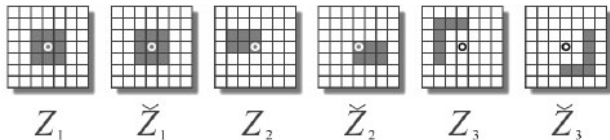
- Continue: 集合的反射集

★ A 的反射集:

$$\hat{A} = \{-a \mid a \in A\}$$

即: A 中所有像素点 关于原点对称翻转后得到的集合.

或者, 在二维空间中, 若 A 是图像中的一个区域, 则 $\hat{A}$ 相当于将 A 绕原点旋转 **180 度**.

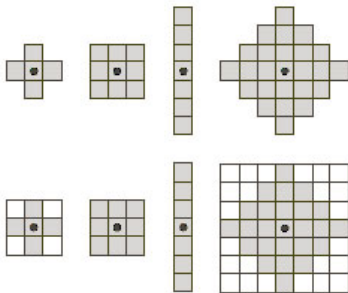


- 结构元素

- ① 数学形态学中的核心工具, 通常是一个小图像(如矩形、圆形或十字形), 用于探测图像中的特定形状或结构;
- ② 其支撑(**support**)描绘了图像平面的邻域像素;
- ③ **SE** 可以是任意形状和尺寸, 任意形式的连通性(**more than 1 piece, have holes**);
- ④ 结构元素的大小和形状会影响处理结果.

Morphological Operations: Structuring Element (SE)

- 下图是结构元素的几个例子:



- ① 第一行: **SE** 的例子; 第二行: 以数组形式表示的 **SE**, 灰色的点为 1, 白色为 0.
- ② 圆点代表 **SE** 的**原点(origin)**, 可在相对其支撑的任意位置。

- 结构元素的平移

- ① 设结构元素为 B ,
- ② 将结构元素平移 $z = (z_1, z_2)$, 得到平移后的结构元素 B_z , 可表示为

$$B_z = \{b + z \mid b \in B\}$$

- i.e., B_p is the translate of B to location p in set of positions.
- ③ 作用: 通过平移结构元素, 可以逐像素地扫描图像, 并对每个像素的邻域进行形态学操作(如膨胀、腐蚀).

- 膨胀运算

- ① **膨胀**: 数学形态学的一种基本操作, 可用于扩展图像中的前景区域;
- ② 通过选择合适的结构元素, 与图像进行卷积操作, 能够填充小孔洞、连接断裂的边缘以及扩展物体的边界;
- ③ 二值图像膨胀运算的标记和定义: $A \oplus B$,

$$A \oplus B = \left\{ z \mid \left(\hat{B} \right)_z \cap A \neq \emptyset \right\}$$

- A : 图像, B : structuring element

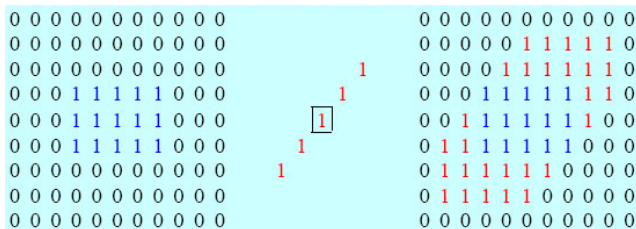
- 关于定义 $A \oplus B$

$$A \oplus B = \left\{ z \mid \left(\hat{B} \right)_z \cap A \neq \emptyset \right\}$$

- ① $\hat{B}$ — 是结构元素的反射(or 关于原点对称);
- ② $(\hat{B})_z$ — 反射后的结构元素平移 z 后的位置;
- ③ 简单理解: 膨胀操作是将结构元素在图像上滑动, 如果结构元素与对象的交集非空, 则将当前中心点标记为前景, 否则标记背景.

Morphological Image Processing: Dilation

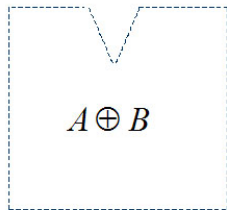
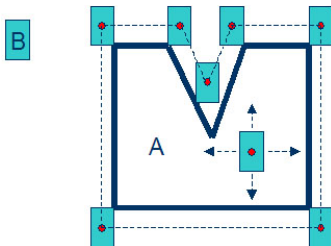
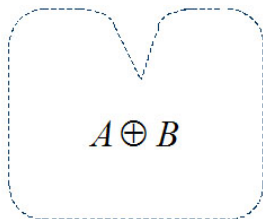
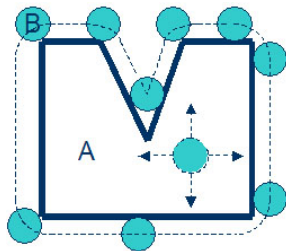
- 下图显示了膨胀算法的原理



- 注意到上图中的 **SE** 是对称的, 原点以框形标记, $\hat{B} = B$.
- 当 **SE** 与标记了 1 的像素存在重合时, **SE** 原点所在的像素标记为 1.

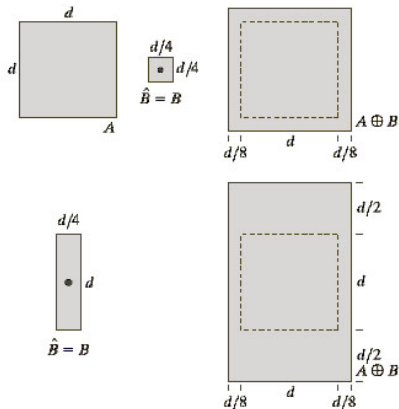
Morphological Image Processing: Dilation

- 用不同的结构元素进行膨胀运算, 结果不同



Morphological Image Processing: Dilation

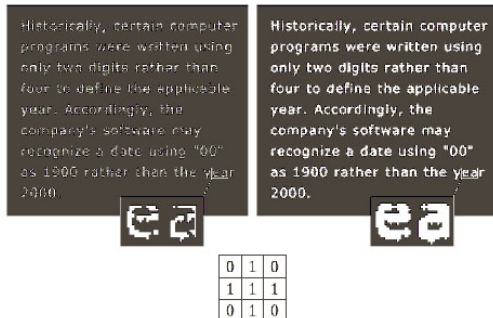
- 膨胀运算的例子(1)



- 图中分别为正方形和长方形的 **SE**, 右图中的虚线是膨胀运算前的轮廓.

Morphological Image Processing: Dilation

- 膨胀运算的例子(2)



- 左图是存在断点的文字，中下是 SE，右图是膨胀后的结果。注意到断点处被连接上了，即通过膨胀运算能连接小的断裂处。

- 膨胀运算的应用场合

- ① 膨胀操作可以去除小的背景噪声;
- ② 可用于扩展物体的边缘;
- ③ 连接断裂区域, 如在字符识别或目标检测中, 用于连接断裂的笔画;
- ④ 在二值图像中填充小的孔洞.

- 膨胀操作的性质

- ① **Commutative(可交换):** $A \oplus B = B \oplus A$.

- ② **Associative(结合性):** $A \oplus (B \oplus C) = (A \oplus B) \oplus C$

- 结合性的应用: Structuring element decomposition

- 如果一个结构元素 B 可表示成 B_1 和 B_2 的膨胀, 即:
 $B = B_1 \oplus B_2$. 则有以下式成立:

$$A \oplus B = A \oplus B_1 \oplus B_2$$

- 结合性的应用: Structuring element decomposition

- 有如下的 5×5 的 **SE**, 可表示成 1×5 和 5×1 的结构元素的膨胀.

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

- 膨胀运算的计算量与结构元素的非零元素数目成正比, 显然 5×5 的 **SE** 有 25 个 1, 而分解后只有 10 个 1.

Morphological Image Processing: Erosion

- 腐蚀: 对二值图像作 “shrink” or “thin”.

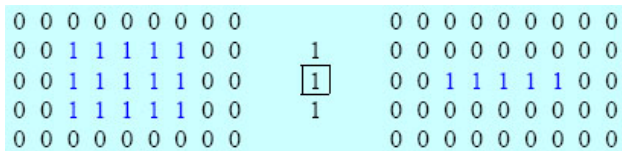
- ① **腐蚀**: 数学形态学的一种基本操作, 可用于缩小或消除图像中的前景对象;
- ② 通过选择合适的结构元素, 与图像进行卷积操作, 能够去除噪声、分离物体和缩小前景区域;
- ③ 标记和定义:

$$A \ominus B = \{z | (B)_z \subseteq A\}$$

- A 代表输入图像, B 是 **SE**;
- $(B)_z$ 为 结构元素 B 平移 $z = (z_1, z_2)$ 后的位置;
- ④ **简单理解**: 结构元素在图像上滑动, 如果当前位置结构元素完全包含于对象, 则将当前中心点标记为前景, 否则标记背景.

Morphological Image Processing: Erosion

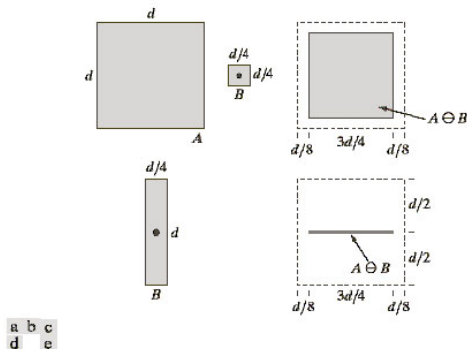
- 下图显示了腐蚀算法的原理



- ① 注意上图中的 **SE** 的原点以框形标记.
- ② 腐蚀检测 **SE** 是否完全包含在前景部分内.

Morphological Image Processing: Erosion

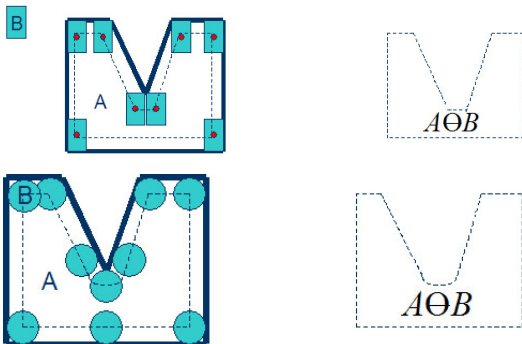
- Erosion运算的例子(1)



- 不同的结构元素导致不同结果: 图 (b) 和 (d), 正方形和细长形状的结构元素; (c) 和 (d), A 被两个结构元素腐蚀的结果(图中的虚线对应原来的边界).

Morphological Image Processing: Erosion

- 用不同的 SE 进行腐蚀运算



- 图中的虚线为结果边界.

- 腐蚀运算的应用场合

- ① 去除小物体: 腐蚀可以去除图像中比结构元素小的孤立点或小物体;
- ② 分离物体: 如果两个物体之间有细小的连接, 腐蚀可以将它们分开;
- ③ 平滑边界: 可以使物体的边界更加平滑;
- ④ 缩小物体: 腐蚀操作会缩小图像中的前景区域.

Morphological Image Processing: opencv-python 实现

- `cv2.dilate()` 和 `cv2.erode()` 的函数原型

opencv的膨胀和腐蚀运算

```
dst = cv2.dilate(src, se, iterations)
```

1. src: 膨胀操作的输入图像，可为任意通道，需是CV_8U、CV_16U、CV_16S等，后续以二值图像；
2. se: 结构元素
3. iterations: 膨胀操作次数

```
dst = cv2.erode(src, kernel, iterations)
```

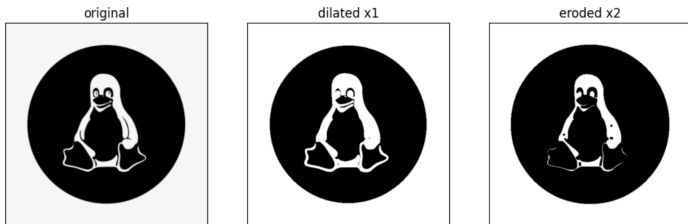
1. 参数基本与dilate()相同

Morphological Image Processing: opencv-python 实现

- 应用示例: dilate x1 接 erode x2 代码和结果

```
ret_thr, bin_im = cv2.threshold(image, thresh=120, maxval=250, type=cv2.THRESH_BINARY)

dil_im = cv2.dilate(bin_im, se, iterations = 1)
ero_im = cv2.erode(dil_im, se, iterations = 2)
```



- 注意: 手臂细纹被填充后, 腐蚀无法复原

Morphological Image Processing: Opening

- 开运算(Opening)

- ① 开运算: 一种结合了腐蚀和膨胀两种基本操作的运算;
- ② 主要作用: 可消除去除图像中的细小物体、平滑物体的边界, 并且保持物体的主要形状不变;

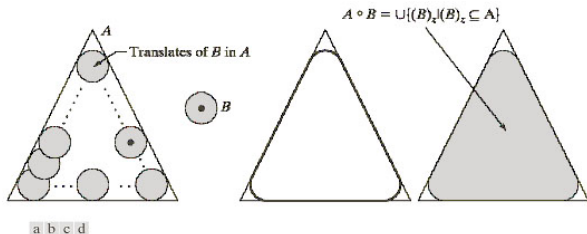
- 定义

$$A \circ B = (A \ominus B) \oplus B$$

- ★ A : 输入图像; B : 结构元素
- ★ 开运算的实质是腐蚀运算后接膨胀运算.

Morphological Image Processing: Opening

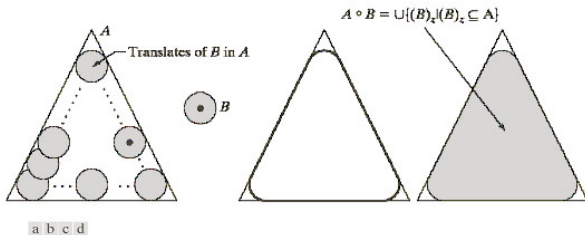
- 下图是开运算的一个示意图



- 1 腐蚀运算的结果是 (a) 图中的虚线.
- 2 图 (c) 的粗线是开运算后的结果轮廓.

Morphological Image Processing: Opening

- 下图是开运算的一个示意图



- ★ 图 (d) 开运算的另一种定义: 结构元素 B 平移的并集, 条件是平移后的 B 是 A 的子集.

$$A \circ B = \cup \{(B)_z \mid (B)_z \subseteq A\}$$

- ★ 比较 腐蚀: $A \ominus B = \{z \mid (B)_z \subseteq A\}$

区别: 腐蚀是结构元素中心点的集合; 开运算是结构元素的集合.

开运算和闭运算：一些性质

• 开运算性质和意义

性质	数学表达	实际意义
幂等性	$(A \circ B) \circ B = A \circ B$	一次应用即稳定，无需重复操作
反扩展性	$A \circ B \subseteq A$	结果不超出原图范围
递增性	$A \subseteq C \implies A \circ B \subseteq C \circ B$	处理结果随输入图像扩展而扩展
数据依赖性	$A \circ B = A \circ (A \cap B)$	只与输入图像中的前景有关
平移不变性	$(A + x) \circ B = (A \circ B) + x$	处理效果与图像位置无关
噪声鲁棒性	无显式公式	有效抑制小尺寸噪声
边界平滑性	无显式公式	平滑凸起边界，保留主要形状

Morphological Image Processing: Closing

- 闭运算(Closing)

- ① 闭运算: 一种结合了腐蚀和膨胀两种基本操作的运算;
- ② 主要作用: 可填充物体内部的小孔洞、连接邻近的物体, 并且保持物体的主要形状不变;

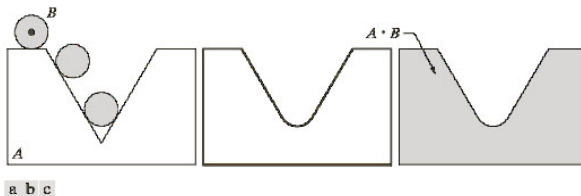
- 定义

$$A \bullet B = (A \oplus B) \ominus B$$

- ★ A : 输入图像; B : 结构元素
- ★ 闭运算的实质是膨胀运算后接腐蚀运算.

Morphological Image Processing: Closing

- 下图是闭运算的一个示意图



- ★ 图 (a) 示意膨胀运算(轮廓将达到图中结构元素中心的连接); (b) 的粗线是闭运算后的结果轮廓; (c) 是阴影表示.

Morphological Image Processing: 开运算和闭运算

• 开运算和闭运算: 对比

特性	开运算	闭运算
顺序	先腐蚀后膨胀	先膨胀后腐蚀
主要作用	去除噪声、分离物体、平滑边界	填充孔洞、连接物体、平滑边界
适用场景	去除细小物体或噪声	填充孔洞或连接断裂物体
对形状的影响	去除细小部分, 保持主要形状	填充孔洞, 保持主要形状

• 应用选择

- ① 需要去除噪声或分离物体, 选择开运算;
- ② 需要填充孔洞或连接物体, 选择闭运算.

开运算和闭运算：一些性质

• 闭运算性质和意义

性质	数学表达	实际意义
幂等性	$(A \bullet B) \bullet B = A \bullet B$	一次应用即稳定，无需重复操作
扩展性	$A \subseteq A \bullet B$	结果包含原图范围
递增性	$A \subseteq C \implies A \bullet B \subseteq C \bullet B$	处理结果随输入图像扩展而扩展
结构元素依赖性	$B_1 \subseteq B_2 \implies A \bullet B_1 \subseteq A \bullet B_2$	结构元素越大，填充的孔洞越多
平移不变性	$(A + x) \bullet B = (A \bullet B) + x$	处理效果与图像位置无关
孔洞填充性	无显式公式	有效填充小尺寸孔洞
边界平滑性	无显式公式	平滑凹陷边界，保留主要形状

- ★ 平移不变性: $(A + x) \bullet B = (A \bullet B) + x$, 即: 对图像进行平移后, 闭运算结果与原图闭运算结果的平移一致

opencv-python 实现的开、闭运算

• cv2.morphologyEx()函数

```
cv2.morphologyEx(src, op, kernel[, dst[, anchor[, iterations[, borderType[, borderValue]]]])
```

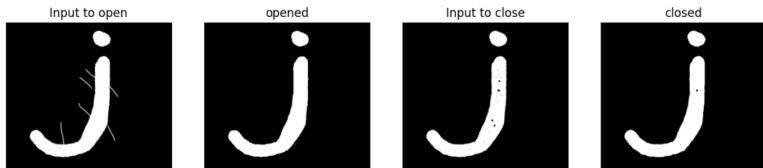
参数说明

- **src**: 输入图像，通常是二值图像（灰度图像也可以，但效果可能不同）。
- **op**: 形态学操作类型，可以是以下值之一：
 - **cv2.MORPH_OPEN**: 开运算（先腐蚀后膨胀）
 - **cv2.MORPH_CLOSE**: 闭运算（先膨胀后腐蚀）
 - **cv2.MORPH_GRADIENT**: 形态学梯度（膨胀图减去腐蚀图）
 - **cv2.MORPH_TOPHAT**: 顶帽运算（原图减去开运算结果）
 - **cv2.MORPH_BLACKHAT**: 黑帽运算（闭运算结果减去原图）
- **kernel**: 结构元素（核），用于定义形态学操作的结构。可以使用 **cv2.getStructuringElement()** 来创建。
- **dst**: 输出图像，与 **src** 大小和类型相同。
- **anchor**: 核的锚点位置，默认是 **(-1, -1)**，表示锚点位于核的中心。
- **iterations**: 操作的迭代次数，默认为 1。
- **borderType**: 边界填充类型，默认为 **cv2.BORDER_CONSTANT**。
- **borderValue**: 边界填充值，默认为 0。

opencv-python 实现的开、闭运算

- cv2.morphologyEx() 函数

```
se = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5), (-1, -1))
# 图像二值化, 开运算
ret_thr, bin_im = cv2.threshold(img_out_noise, 160, 255, cv2.THRESH_BINARY)
bin_opened = cv2.morphologyEx(bin_im, cv2.MORPH_OPEN, se)
# 二值化, 闭运算
ret_thr, bin_im = cv2.threshold(img_in_noise, 160, 255, cv2.THRESH_BINARY)
bin_closed = cv2.morphologyEx(bin_im, cv2.MORPH_CLOSE, se)
```



由膨胀和腐蚀导出的运算：形态学梯度

- 形态学梯度: 定义

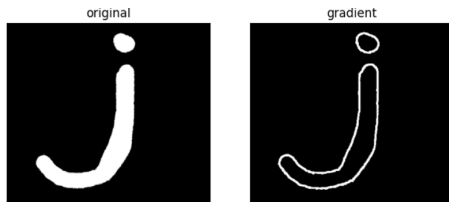
$$\text{Gradient} = (A \oplus B) - (A \ominus B)$$

- ★ A — 输入图像; B — 结构元素;
- ★ 膨胀会使图像中的前景扩张, 腐蚀会使前景收缩; 通过计算膨胀和腐蚀的差, 可以得到物体的边缘信息.

- cv2.morphologyEx()函数, op 选择 “cv2.MORPH_GRADIENT”

```
# 5*5 大的结构元素导致粗的边缘
se = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5), (-1, -1))
# 二值化和梯度运算
ret_thr, bin_im = cv2.threshold(image, thresh=120, maxval=255, type=cv2.THRESH_BINARY)
bin_grad = cv2.morphologyEx(bin_im, cv2.MORPH_GRADIENT, se)
```

★ 结果示例



由开运算导出的运算: TOPHAT

- 顶帽运算: 定义

$$\text{Top_Hat} = A - (A \circ B)$$

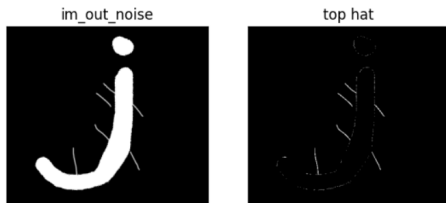
- ★ A — 输入图像; B — 结构元素;
- ★ $\circ$ 代表开运算;
- ★ 先腐蚀后膨胀会平滑图像并去除细小的前景区域, 顶帽运算的结果: 原始图像中那些被开运算去除的细小前景区域.

opencv-python 实现的顶帽运算

- `cv2.morphologyEx()` 函数, `op` 选 “`cv2.MORPH_TOPHAT`”

```
# 5*5 大的结构元素, 留下大的小目标  
se = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5), (-1, -1))  
# 图像二值化, Top hat  
ret_thr, bin_im = cv2.threshold(img_out_noise, 160, 255, cv2.THRESH_BINARY)  
bin_top = cv2.morphologyEx(bin_im, cv2.MORPH_TOPHAT, se)
```

★ 结果示例



由闭运算导出的运算: BLACKHAT

- 黑帽运算: 定义

$$\text{Black_Hat} = (A \bullet B) - A$$

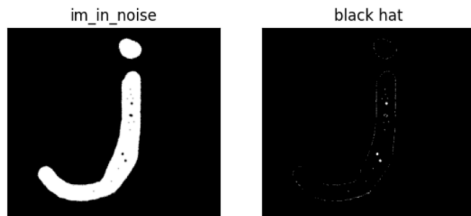
- ★ A — 输入图像; B — 结构元素;
- ★ $\bullet$ 代表闭运算;
- ★ 先膨胀后腐蚀会平滑图像并填充小的背景区域, 黑帽运算的结果: 原始图像中那些被闭运算填充的细小背景区域.

opencv-python 实现的黑帽运算

- `cv2.morphologyEx()` 函数, 选 “`cv2.MORPH_BLACKHAT`”

```
se = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5), (-1, -1))  
# 图像二值化, blacks hat  
ret_thr, bin_im = cv2.threshold(img_in_noise, 160, 255, cv2.THRESH_BINARY)  
bin_black = cv2.morphologyEx(bin_im, cv2.MORPH_BLACKHAT, se)
```

★ 结果示例



- 灰度图像的形态学滤波: 概述

- ① 灰度图像 $f(x, y)$ 可取 0, 1 以外的值, 无法用集合表示.
- ② 灰度图像的形态学滤波与二值图像的形态学运算有所不同, 需要用不同的形式来描述.

灰度图像的形态学滤波

- 灰度图像的形态学滤波: 腐蚀

- ① **灰度腐蚀**: 可用于减小灰度图像中的亮区域并扩大暗区域;
- ② 有几种不同的定义, **opencv** 实现的定义如下:
 - ★ 设输入灰度图像为 $X(x, y)$, $0 \leq x \leq M - 1$, $0 \leq y \leq N - 1$, 结构元素为 B (小的矩阵或掩模) .
 - ★ 结构元素对 $X(x, y)$ 腐蚀运算的结果为 $E(x, y)$, 其定义如下:

$$\begin{aligned} E(x, y) &= (X \ominus B)(x, y) \\ &= \min_{s, t \in B} \{X(x + s, y + t)\} \end{aligned}$$

- (x, y) 代表图像中的像素坐标; (s, t) 代表结构元素中的坐标偏移

灰度图像的形态学滤波

- 灰度图像的形态学滤波: 腐蚀例子

- ★ 设有如下的灰度图像和结构元素

$$\begin{pmatrix} 11 & 10 & 11 & 9 \\ 10 & 3 & 6 & 6 \\ 11 & 8 & 7 & 3 \\ 3 & 6 & 4 & 9 \end{pmatrix}$$

图像

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

结构元素

- ★ 以计算(2,1) 位置(红色标记的数字8)为例: 结构元素的锚点与该位置对齐, 取图像中与结构元素中为 1 点所对应的所有像素值的最小值作为腐蚀结果.
- ★ i.e., 该像素的腐蚀结果为 3

- opencv-python 的 `cv2.erode()`

在 OpenCV 中, `cv2.erode()` 函数的原型如下:

```
python
```

```
cv2.erode(src, kernel, dst=None, anchor=None, iterations=1, borderType=None, borderValue=None)
```

- **src**: 输入图像 (可以是灰度图像或二值图像)。
- **kernel**: 结构元素 (通常通过 `cv2.getStructuringElement()` 创建)。
- **iterations**: 腐蚀操作的迭代次数, 默认为 1。
- **borderType**: 边界填充类型 (默认为 `cv2.BORDER_CONSTANT`)。
- **borderValue**: 边界填充值 (默认为 0)。

灰度图像的形态学滤波

- 灰度图像的形态学滤波: 膨胀

① **灰度膨胀**: 与灰度腐蚀相反, 它用于扩大图像中的亮区域并缩小暗区域;

② 有几种不同的定义, **opencv** 实现的定义如下:

★ 设输入灰度图像为 $X(x, y)$, $0 \leq x \leq M - 1$, $0 \leq y \leq N - 1$, 结构元素为 B (小的矩阵或掩模) .

★ 结构元素对 $X(x, y)$ 膨胀运算的结果为 $D(x, y)$, 其定义如下:

$$\begin{aligned} D(x, y) &= (X \oplus B)(x, y) \\ &= \max_{s, t \in B} \{X(x + s, y + t)\} \end{aligned}$$

- (x, y) 代表图像中的像素坐标; (s, t) 代表结构元素中的坐标偏移

灰度图像的形态学滤波

- 灰度图像的形态学滤波: 膨胀例子

- ★ 设有如下的灰度图像和结构元素

$$\begin{pmatrix} 11 & 10 & 11 & 9 \\ 10 & 3 & 6 & 6 \\ 11 & 8 & 7 & 3 \\ 3 & 6 & 4 & 9 \end{pmatrix}$$

图像

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

结构元素

- ★ 以计算(2,1) 位置(红色标记的数字8)为例: 结构元素的锚点与该位置对齐, 取图像中与结构元素中为 1 点所对应的所有像素值的最大值作为膨胀结果.
- ★ i.e., 该像素的膨胀结果为 11

灰度图像的形态学滤波

• opencv-python 的 cv2.dilate()

在 OpenCV 中, `cv2.dilate()` 函数用于实现灰度膨胀。其原型如下:

python

复制

```
cv2.dilate(src, kernel, dst=None, anchor=None, iterations=1, borderType=None, borderValue=None)
```

- **src**: 输入图像 (可以是灰度图像或二值图像)。
- **kernel**: 结构元素 (通常通过 `cv2.getStructuringElement()` 创建)。
- **iterations**: 膨胀操作的迭代次数, 默认为 1。
- **borderType**: 边界填充类型 (默认为 `cv2.BORDER_CONSTANT`)。
- **borderValue**: 边界填充值 (默认为 0)。

● 灰度图像的形态学滤波: 小结

- ① 如果结构元素的所有像素都为正值, 则膨胀使图像变亮, 腐蚀使图像变暗;
- ② 膨胀使黑色细节减少或去除, 腐蚀使亮的细节减少或去除, 具体的量取决于结构元素的形状和图像的局部值.
- ③ 主要应用:
 - ★ 灰度膨胀 去除亮噪声, 分离物体;
 - ★ 灰度腐蚀 填充暗噪声, 连接物体;